

UNIT 3: MACHINE LEARNING FOUNDATIONS & SUPER- VISED LEARNING

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Publication-Grade Theory Guide	Course Code: PCC-302-IT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 3 based on SPPU end-semester examination trends.

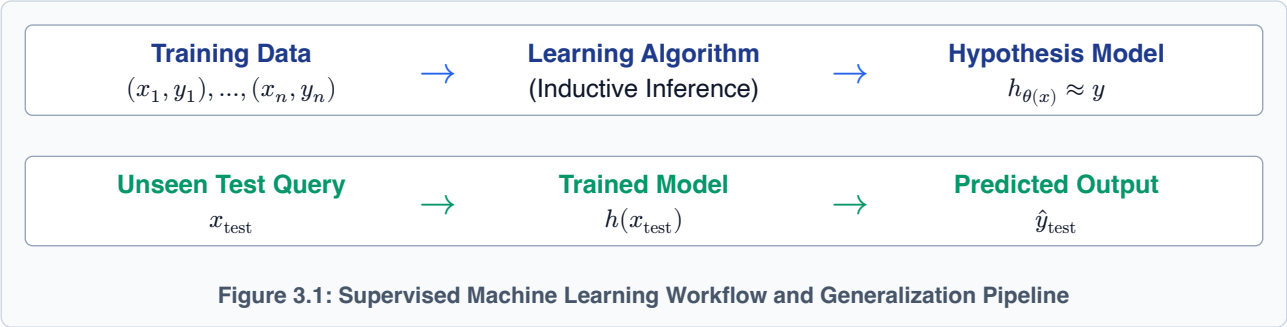
Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
ML Foundations & Paradigms	Mitchell's T-P-E framework, Supervised vs Unsupervised vs Semi-supervised vs RL, Scales of feature measurement, Hypothesis space, Version space, Inductive bias, VC-Dimension.	High Priority (6-8 Marks Qs)
Linear & Multiple Regression	Hypothesis formulation, Ordinary Least Squares (OLS) Normal Equations, Batch/Stochastic Gradient Descent, Mean Squared Error cost function.	High Priority (7-9 Marks Qs)
Regularization & Logistic Regression	Overfitting vs Underfitting, L2 Ridge vs L1 Lasso penalties, Logistic sigmoid function, Odds ratio, Logit, Cross-entropy loss, Multiclass (OvR, OvO).	High Priority (8-10 Marks Qs)
k-Nearest Neighbors (k-NN)	Instance-based lazy learning, Distance metrics (Euclidean, Manhattan, Minkowski, Cosine), Choice of k , Weighted k-NN, Regression with k-NN.	Medium-High Priority (6-8 Marks Qs)
Naive Bayes Classifier	Bayes' Theorem, Conditional independence assumption, Maximum A Posteriori (MAP), Prior/Likelihood/Posterior, Laplace Smoothing, NB variants.	High Priority (6-8 Marks Qs)
Decision Tree Learning	Tree anatomy (Root, Internal, Leaf), ID3 Information Gain & Entropy, CART Gini Impurity, C4.5 Gain Ratio, Pre-pruning vs Post-pruning, Missing values.	High Priority (8-12 Marks Qs)
Support Vector Machines (SVM)	Maximum margin hyperplanes, Support vectors, Hard vs Soft margin, Slack variables ξ_i , Regularization C , Kernel Trick (Polynomial, RBF, Sigmoid).	High Priority (8-10 Marks Qs)
Model Evaluation & Validation	Confusion Matrix (TP, TN, FP, FN), Precision, Recall, F1-Score, Specificity, ROC-AUC, Bias-Variance decomposition, K-Fold, LOOCV, Stratified CV.	High Priority (8-10 Marks Qs)

Machine Learning Foundations & Learning Paradigms

Formal Definitions of Machine Learning

Machine Learning (ML) is a core subfield of Artificial Intelligence focused on developing computational algorithms that iteratively improve their task execution performance through automated statistical inference and experiential data, without requiring explicit rule-based hardcoding.

1. **Arthur Samuel’s Definition (1959):** Field of study that gives computers the ability to learn without being explicitly programmed.
2. **Tom Mitchell’s Operational Definition (1997):** A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .
 - **Task (T):** The specific operational objective to be accomplished (e.g., classifying whether an incoming email is spam or ham, predicting real-estate prices, medical diagnosis).
 - **Experience (E):** The historical training dataset or interaction log supplied to the algorithm (e.g., a collection of 50,000 labeled email messages).
 - **Performance Measure (P):** The quantitative metric used to assess the effectiveness of the learned model (e.g., classification accuracy, precision, mean squared error).



Fundamental Concepts: Hypothesis Space, Version Space & Inductive Bias

1. **Hypothesis Space ($\mathcal{H}$):** The set of all possible candidate functions or models $h \in \mathcal{H}$ that the learning algorithm can entertain to approximate the unknown true target function $c : X \rightarrow Y$.
2. **Target Concept (c):** The underlying true mapping $c(x)$ that generated the ground-truth labels in the domain.
3. **Consistent Hypothesis:** A hypothesis h is consistent with a set of training examples D if and only if $h(x) = c(x) = y$ for every sample $(x, y) \in D$.
4. **Version Space ($VS_{\mathcal{H}, D}$):** The subset of all hypotheses in $\mathcal{H}$ that are completely consistent with all training instances in dataset D :

$$VS_{\mathcal{H}, D} = \{h \in \mathcal{H} \mid \forall (x, y) \in D, h(x) = y\}$$

5. **Inductive Bias:** The set of explicit prior assumptions and preferences that a learner incorporates beyond the raw training data to generalize to previously unseen instances. Without inductive bias, a learner cannot extrapolate beyond observed samples (Futility of Bias-Free Learning).
 - **Restriction Bias (Language Bias):** Constraining the hypothesis space to a specific mathematical family (e.g., linear hyperplanes in Linear Regression/SVM).
 - **Preference Bias (Search Bias):** Preferring certain hypotheses over others within an unrestricted space (e.g., Occam's Razor in Decision Trees: preferring smaller trees with higher Information Gain).
6. **Vapnik-Chervonenkis (VC) Dimension:** A fundamental measure of the capacity or expressive richness of a space of hypotheses $\mathcal{H}$. The VC dimension $VC(\mathcal{H})$ is the maximum number of points d that can be **shattered** (separated into all possible 2^d binary label assignments) by $\mathcal{H}$.
 - For linear classifiers in d -dimensional Euclidean space $\mathbb{R}^d$, $VC(\mathcal{H}) = d + 1$.
 - VC dimension provides theoretical distribution-free bounds on the sample complexity needed for Probably Approximately Correct (PAC) generalization.

Scales of Feature Measurement in Machine Learning

In machine learning feature engineering, variables are classified into four hierarchical measurement scales (Stevens' Taxonomy), which dictate admissible mathematical transformations:

Scale	Mathematical Properties	Admissible Operations	ML Example & Encoding
Nominal	Categorical, discrete labels; identity only, no natural ordering.	Equality ($=$ or $\neq$, Mode). No arithmetic operations.	Blood type (A, B, AB, O), City name. Encoded via One-Hot Encoding.
Ordinal	Ranked categories; ordered sequence with undefined interval distance.	Comparison ($<$, $>$, Median, Percentiles). No addition.	Education level (High School, BS, MS, PhD), Customer rating (1-5 stars). Encoded via Ordinal Encoding.
Interval	Quantitative numeric; meaningful partial orderings, arbitrary/shifted zero.	Addition and subtraction ($+$, $-$, Mean, Variance). No ratio multiplication.	Temperature in Celsius/Fahrenheit, Calendar dates. Standard scaling applicable.
Ratio	Quantitative numeric; meaningful partial orderings and true absolute zero.	All arithmetic operations ($+$, $-$, $*$, $\div$, Geometric Mean).	Salary, Distance, Weight, Age, Revenue. Normalized via Min-Max or Standard scaling.

Source: Alpaydin, Ch 1-2, p. 1-38 | Source: Marsland, Ch 1-2, p. 1-25 | Source: Russell & Norvig, Ch 19, p. 671-693

Regression Models: Linear, Multivariate, Regularization & Logistic

Simple Linear Regression

Simple Linear Regression models the linear functional relationship between a single independent scalar explanatory variable x and a dependent continuous target response y .

1. Hypothesis Formulation:

$$h_{\theta(x)} = \theta_0 + \theta_1 x$$

where θ_0 is the intercept term (bias) and θ_1 is the slope coefficient (weight).

2. Mean Squared Error (MSE) Cost Function: The parameters are estimated by minimizing the residual sum of squared partialerences over m training samples:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta(x^{(i)})} - y^{(i)})^2$$

3. Ordinary Least Squares (OLS) Closed-Form Normal Equations: Setting partial derivatives $\partial \frac{J}{\partial \theta_0} = 0$ and $\partial \frac{J}{\partial \theta_1} = 0$ yields:

$$\theta_1 = \frac{\sum_{i=1}^m (x^{(i)} - |x|)(y^{(i)} - |y|)}{\sum_{i=1}^m (x^{(i)} - |x|)^2} = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$$

$$\theta_0 = |y| - \theta_1 |x|$$

4. Gradient Descent Optimization Algorithm: An iterative first-order optimization algorithm that updates parameters in the direction of steepest descent:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta(x^{(i)})} - y^{(i)}) x_j^{(i)}$$

where $\alpha > 0$ is the **learning rate**. If α is too small, convergence is slow; if α is too large, gradient descent can overshoot the minimum and diverge.

Multiple Linear Regression (Multivariate Model)

Multiple Linear Regression generalizes simple regression to d explanatory input features $\mathbf{x} = [x_1, x_2, \dots, x_d]^T \in \mathbb{R}^d$:

1. Vectorized Model Representation:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d = \boldsymbol{\theta}^T \mathbf{x}$$

where $\mathbf{x} = [1, x_1, x_2, \dots, x_d]^T$ (with dummy intercept feature $x_0 = 1$) and $\boldsymbol{\theta} = [\theta_0, \theta_1, \dots, \theta_d]^T$.

2. Matrix-Vector Formulation: For dataset of m instances with design matrix $\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$ and target vector $\mathbf{y} \in \mathbb{R}^m$:

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & \dots & x_{1d} \\ 1 & x_{21} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & \dots & x_{md} \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}, \quad \boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{pmatrix}$$

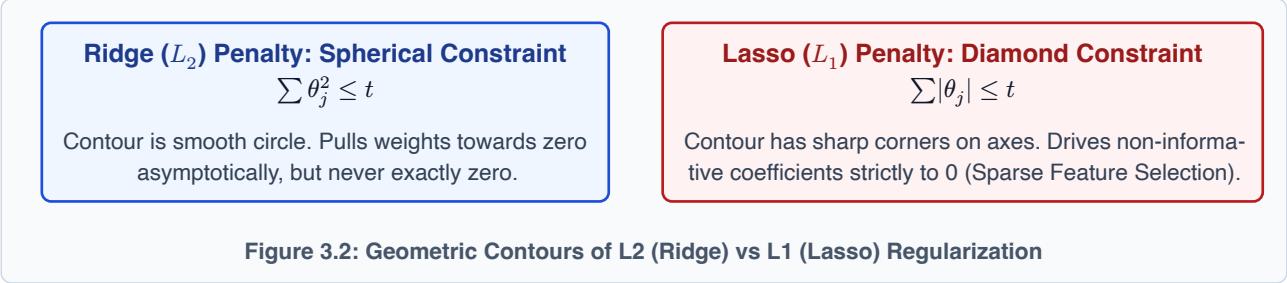
3. Closed-Form Normal Equation Solution:

$$\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- Requires $(\mathbf{X}^T \mathbf{X})$ to be non-singular (invertible). Invertibility fails if features are linearly dependent (multicollinearity) or if $d > m$.
- Computational complexity of matrix inversion is $\mathcal{O}(d^3)$, making Gradient Descent preferred when $d > 10^4$.

Regularization in Regression: Ridge vs. Lasso

When training models on high-dimensional data, unconstrained OLS often suffers from high variance (overfitting) and numerical instability. **Regularization** introduces a penalty term to the loss function to constrain coefficient magnitudes.



1. Ridge Regression (L_2 Regularization / Tikhonov):

$$J_{\text{Ridge}(\theta)} = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^d \theta_j^2 = \text{MSE} + \lambda \|\theta\|_2^2$$

- Analytical solution: $\theta = (X^T X + \lambda I)^{-1} X^T y$ (always invertible for $\lambda > 0$).
- Shrinks collinear coefficients proportionally, mitigating multicollinearity.

2. Lasso Regression (L_1 Regularization - Least Absolute Shrinkage and Selection Operator):

$$J_{\text{Lasso}(\theta)} = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^d |\theta_j| = \text{MSE} + \lambda \|\theta\|_1$$

- Generates sparse models by zeroing out non-essential coefficients, acting as automated embedded feature selection.

Parameter	Ordinary Least Squares	Ridge Regression (L_2)	Lasso Regression (L_1)
Penalty Term	None ($\lambda = 0$).	$\lambda \sum_{j=1}^d \theta_j^2$ (Squared Euclidean norm).	$\lambda \sum_{j=1}^d \theta_j $ (Manhattan norm).
Weight Shrinkage	Unbounded, prone to exploding on collinear data.	Shrinks weights close to zero, never strictly zero.	Shrinks weights strictly to zero (exact sparsity).
Feature Selection	No (retains all features).	No (retains all features with reduced weights).	Yes (acts as automatic feature selection).
Closed-Form Solution	$\theta = (X^T X)^{-1} X^T y$	$\theta = (X^T X + \lambda I)^{-1} X^T y$	No analytical form; solved via coordinate descent.
Primary Use Case	Low dimensional data ($m \gg d$) with no collinearity.	Data with severe multicollinearity and dense weights.	High-dimensional data where only few features matter.

Logistic Regression for Binary Classification

Despite its name, Logistic Regression is a fundamental linear model for **binary classification** that models the conditional probability $P(Y = 1 \mid x)$.

1. Odds Ratio and Logit Transformation:

- **Odds:** Ratio of the probability of occurrence to non-occurrence: $\text{Odds} = \frac{p}{1-p}$.
- **Logit Function (Log-Odds):** Maps probabilities from $(0, 1)$ to $(-\infty, +\infty)$:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right) = \theta^T x = \theta_0 + \theta_1 x_1 + \dots + \theta_d x_d$$

2. Sigmoid (Logistic) Activation Function: Inverting the logit yields the hypothesis function:

$$h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

- Output range: $0 < h_{\theta}(x) < 1$, representing $P(y = 1 \mid x; \theta)$.
- Derivative property: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

3. Decision Boundary: For threshold $p = 0.5$:

$$\hat{y} = \begin{cases} 1 & \text{if } \theta^T x \geq 0 \\ 0 & \text{if } \theta^T x < 0 \end{cases}$$

4. Binary Cross-Entropy (Log-Loss) Cost Function: Mean squared error produces non-convex landscape for logistic regression; cross-entropy guarantees convexity:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \ln(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \ln(1 - h_{\theta}(x^{(i)}))]$$

5. Parameter Update Rule:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

6. Multiclass Classification Extensions:

- **One-vs-Rest (OvR / One-vs-All):** For K classes, train K separate binary classifiers where class k is positive and all other $K - 1$ classes are negative. Prediction: $\hat{y} = \arg \max_k h_{\theta}^{(k)}(x)$.
- **One-vs-One (OvO):** Train $\frac{K(K-1)}{2}$ binary classifiers for every pair of classes (j, k) . Prediction by majority voting among all pairwise classifiers.

Parameter	Linear Regression	Logistic Regression
Output Nature	Continuous numeric values ($y \in \mathbb{R}$).	Discrete categorical probabilities ($p \in [0, 1]$).
Hypothesis Function	$h_{\theta}(x) = \theta^T x$ (Linear equation).	$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$ (Sigmoid).
Cost Function	Mean Squared Error (MSE / Residual Sum of Squares).	Binary Cross-Entropy Loss (Log-Loss).
Underlying Goal	Fit the optimal line/hyperplane minimizing distance to points.	Find the optimal decision hyperplane separating classes.
Thresholding	Not required (direct quantitative prediction).	Required (e.g., threshold $\geq 0.5 \rightarrow$ Class 1, else 0).

Source: Alpaydin, Ch 5 & 10, p. 103-108, 228-235 | Source: Marsland, Ch 3, p. 64-70 | Source: Russell & Norvig, Ch 19, p. 694-706

Classification Algorithms: k-NN, Naive Bayes, Decision Trees & SVM

k-Nearest Neighbors (k-NN)

k-Nearest Neighbors is a non-parametric, **lazy learning (instance-based)** algorithm that defers all generalization computation until a query instance is presented.

1. Algorithm Mechanics:

- Store all m training instances $(x^{(i)}, y^{(i)})$.
- Given a query test sample x_q , compute distance $d(x_q, x^{(i)})$ to all training points.
- Select the k training points with smallest distances ($N_{k(x_q)}$).

- **Classification Decision:** Majority voting among the k neighbors:

$$\hat{y} = \arg \max_{c \in C} \sum_{i \in N_k(x_q)} \mathbb{I}(y^{(i)} = c)$$

- **Distance-Weighted Classification:** Closer neighbors contribute higher voting weight:

$$w_i = \frac{1}{d(x_q, x^{(i)})^2 + \varepsilon}, \quad \hat{y} = \arg \max_{c \in C} \sum_{i \in N_k(x_q)} w_i \cdot \mathbb{I}(y^{(i)} = c)$$

- **k-NN Regression:** Output the arithmetic or distance-weighted mean:

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(x_q)} y^{(i)} \quad \text{or} \quad \hat{y} = \frac{\sum w_i y^{(i)}}{\sum w_i}$$

2. Common Distance Metrics in Feature Space:

- **Euclidean Distance (L_2 norm):** $d(x, z) = \sqrt{\sum_{j=1}^d (x_j - z_j)^2}$
- **Manhattan Distance (L_1 norm / City Block):** $d(x, z) = \sum_{j=1}^d |x_j - z_j|$
- **Minkowski Distance (L_p generalized norm):** $d(x, z) = \left(\sum_{j=1}^d |x_j - z_j|^p \right)^{\frac{1}{p}}$ ($p = 1$ Manhattan, $p = 2$ Euclidean).
- **Cosine Similarity / Distance:** $\text{CosSim}(x, z) = \frac{x \cdot z}{\|x\| \cdot \|z\|}$, $d_{\text{cosine}} = 1 - \text{CosSim}$.

3. Effect of Hyperparameter k :

- Small k (e.g., $k = 1$): Extremely complex decision boundary, highly sensitive to noise, low bias, high variance (**Overfitting**).
- Large k (e.g., $k = m$): Oversimplified decision boundary (predicts dominant majority class), high bias, low variance (**Underfitting**).
- Optimal k is typically an odd number (to prevent ties in binary classification) selected via cross-validation.

4. Curse of Dimensionality:

In high-dimensional spaces ($d \gg 1$), sample volume grows exponentially, making all points equidistant from each other and rendering standard distance metrics non-discriminative.

Naive Bayes Classifier

Naive Bayes is a probabilistic generative classifier grounded in **Bayes' Theorem** combined with the **naive conditional independence assumption**.

1. Bayes' Theorem Formulation:

$$P(Y = c | x) = \frac{P(x | Y = c) \cdot P(Y = c)}{P(x)} = \frac{P(x_1, x_2, \dots, x_d | c) \cdot P(c)}{P(x_1, x_2, \dots, x_d)}$$

- **Posterior Probability $P(Y = c | x)$:** Probability of class c given feature vector x .
- **Prior Probability $P(Y = c)$:** Baseline probability of class c in the training corpus ($P(c) = \frac{N_c}{N}$).
- **Likelihood $P(x | Y = c)$:** Probability of observing feature vector x within class c .
- **Evidence $P(x)$:** Normalizing marginal probability constant across all classes ($\sum_k P(x | c_k) P(c_k)$).

2. Class-Conditional Independence Assumption:

Assumes that each feature x_j is conditionally independent of every other feature x_k given class c :

$$P(x | c) = P(x_1, x_2, \dots, x_d | c) = \prod_{j=1}^d P(x_j | c)$$

3. Maximum A Posteriori (MAP) Decision Rule:

$$\hat{y} = \arg \max_{c \in C} \left[P(c) \prod_{j=1}^d P(x_j | c) \right]$$

In computational implementations, log-probabilities are summed to prevent numerical floating-point underflow:

$$\hat{y} = \arg \max_{c \in C} \left[\ln P(c) + \sum_{j=1}^d \ln P(x_j | c) \right]$$

4. **Zero-Frequency Problem & Laplace (Additive) Smoothing:** If a feature value x_j never co-occurs with class c in the training data, $P(x_j | c) = 0$, causing the entire product to collapse to zero. **Laplace Smoothing** adds pseudo-counts:

$$\hat{P}(x_j = v | c) = \frac{N_{c,v} + \alpha}{N_c + \alpha \cdot |V|}$$

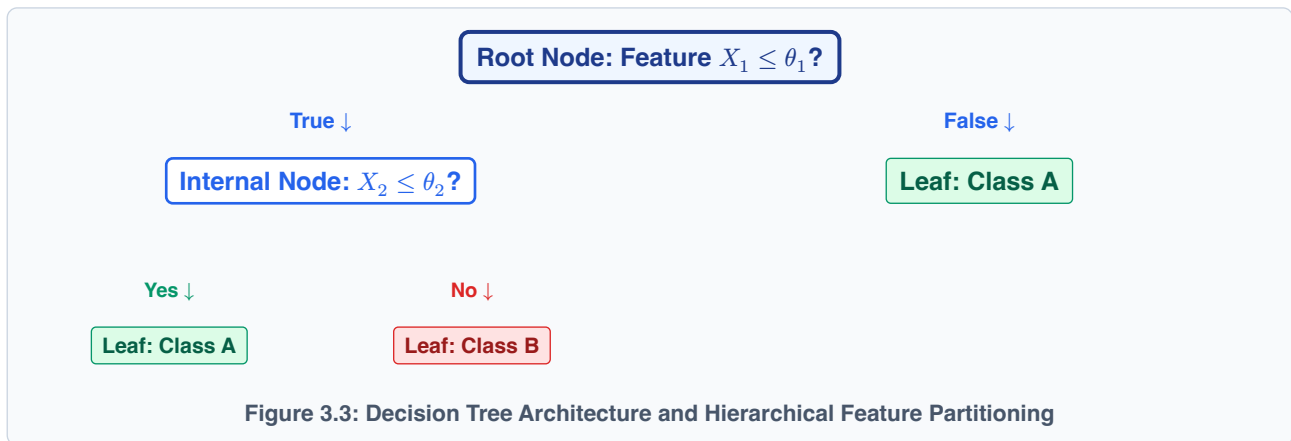
where $|V|$ is the feature vocabulary size and $\alpha = 1$ is Laplace smoothing parameter ($\alpha < 1$ is Lidstone smoothing).

5. **Common Naive Bayes Variants:**

- **Gaussian Naive Bayes:** Continuous real-valued features modeled via normal distribution: $P(x_j | c) = \frac{1}{\sqrt{2\pi\sigma_{c,j}^2}} \exp\left(-\frac{(x_j - \mu_{c,j})^2}{2\sigma_{c,j}^2}\right)$.
- **Multinomial Naive Bayes:** Discrete word counts / term frequencies (e.g., text categorization, spam filtering).
- **Bernoulli Naive Bayes:** Binary boolean feature occurrences (0 or 1) (e.g., presence or absence of keywords).

Decision Tree Learning (ID3, CART & C4.5)

A Decision Tree is a hierarchical non-parametric model that recursively partitions the feature space into axis-aligned hyper-rectangles using greedy heuristic splitting.



1. **Decision Tree Terminology:**

- **Root Node:** The topmost decision node containing the full undivided dataset.
- **Internal (Decision) Node:** Intermediate test condition on a specific feature.
- **Branch (Edge):** Outcome path corresponding to a feature test.
- **Leaf (Terminal) Node:** Node holding the final class prediction or continuous value.

2. **Splitting Criteria:**

- **Shannon Entropy ($H(S)$):** Measure of impurity or disorder in sample set S containing proportions p_i of class i :

$$H(S) = - \sum_{i=1}^C p_i \log_2(p_i)$$

- Pure node ($p_1 = 1$): $H(S) = 0$. Maximum impurity (binary equal split $p_1 = 0.5, p_2 = 0.5$): $H(S) = 1.0$.
- **Information Gain (ID3 Algorithm):** Expected reduction in entropy caused by partitioning on attribute A :

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \left(\frac{|S_v|}{|S|} \right) H(S_v)$$

- **Limitation:** Heavily biased toward attributes with many distinct values (e.g., CustomerID, Date).
- **Gain Ratio (C4.5 Algorithm):** Normalizes Information Gain by Split Information to penalize multi-valued attributes:

$$\text{SplitInfo}_{A(S)} = - \sum_{v \in \text{Values}(A)} \left(\frac{|S_v|}{|S|} \right) \log_2 \left(\frac{|S_v|}{|S|} \right), \quad \text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}_{A(S)}}$$

- **Gini Impurity (CART Algorithm - Classification and Regression Trees):** Measures the probability of misclassifying a randomly chosen element:

$$\text{Gini}(S) = 1 - \sum_{i=1}^C p_i^2$$

- Pure node: Gini = 0. Maximum binary impurity: Gini = 0.5.
- Computationally faster than Entropy because it avoids logarithmic operations.

3. Tree Pruning (Mitigating Overfitting):

- **Pre-Pruning (Early Stopping):** Stop growing tree if sample count falls below threshold (`min_samples_split`), maximum depth is reached (`max_depth`), or impurity drop is below ϵ .
- **Post-Pruning (Cost-Complexity Pruning):** Grow full tree until leaves are pure, then iteratively collapse non-essential subtrees minimizing cost: $R_{\alpha}(T) = R(T) + \alpha |T|$.

4. Handling Missing Values:

Impute with modal/mean value within the partition or distribute sample fractionally across all sub-branches proportionally to child frequencies.

Support Vector Machines (SVM)

Support Vector Machine is a supervised maximum-margin classifier that constructs an optimal separating hyperplane in a high-dimensional feature space.

Positive Hyperplane

$$w^T x + b = +1$$

(Positive Support Vectors ●)

Optimal Decision Boundary

$$w^T x + b = 0$$

Margin Width $\gamma = \frac{2}{\|w\|}$

Negative Hyperplane

$$w^T x + b = -1$$

(Negative Support Vectors ■)

Figure 3.4: Support Vector Machine Maximum Margin Geometry and Support Vectors

1. Hyperplane and Functional Margin:

- Decision Hyperplane: $w^T x + b = 0$ where w is the normal weight vector and b is the bias offset.
- For linearly separable classes $y_i \in \{-1, +1\}$:

$$\begin{cases} w^T x^{(i)} + b \geq +1 & \text{for } y^{(i)} = +1 \\ w^T x^{(i)} + b \leq -1 & \text{for } y^{(i)} = -1 \end{cases} \Rightarrow y^{(i)}(w^T x^{(i)} + b) \geq 1$$

2. Geometric Margin and Hard-Margin Optimization:

- Total geometric margin between bounding hyperplanes is $\gamma = \frac{2}{\|w\|}$.
- Maximizing margin $\frac{2}{\|w\|}$ is equivalent to solving the convex quadratic optimization problem:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad \forall i = 1, \dots, m$$

- Support Vectors:** The subset of training instances that lie exactly on the marginal bounding hyperplanes ($y^{(i)}(w^T x^{(i)} + b) = 1$). All other data points can be deleted without altering the decision boundary.
- Soft-Margin SVM (Non-Separable Data & Slack Variables):** To handle non-linearly separable or noisy distributions, slack variables $\xi_i \geq 0$ allow margin violations:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i \quad \text{subject to} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- **Regularization Parameter C :** Governs the trade-off between maximizing margin width and minimizing training classification errors.
 - Large C : Strict penalty on errors, narrow margin, low bias, high variance (**Overfitting risk**).
 - Small C : Tolerates margin violations, wide margin, high bias, low variance (**Underfitting risk**).

5. The Kernel Trick (Non-Linear SVM):

When data is non-linearly separable in input space $\mathbb{R}^d$, map it to higher-dimensional feature space $\Phi(x)$ where it becomes linearly separable. **Mercer's Theorem** allows computing inner products in high-dimensional space without explicitly evaluating $\Phi(x)$:

$$K(x, z) = \langle \Phi(x), \Phi(z) \rangle$$

- **Linear Kernel:** $K(x, z) = x^T z$
- **Polynomial Kernel:** $K(x, z) = (x^T z + c)^d$
- **Radial Basis Function (RBF / Gaussian Kernel):** $K(x, z) = \exp(-\gamma \|x - z\|^2)$ where $\gamma = \frac{1}{2\sigma^2}$. Maps data to infinite-dimensional Hilbert space.
- **Sigmoid Kernel:** $K(x, z) = \tanh(\alpha x^T z + c)$

Source: Alpaydin, Ch 8, 9, 13, p. 168-195, 318-335 | Source: Marsland, Ch 8 & 12, p. 169-188, 249-265 | Source: Russell & Norvig, Ch 19, p. 707-720

Model Evaluation, Validation & Diagnostic Techniques

Confusion Matrix & Classification Performance Metrics

For binary classification, model predictions against actual ground-truth labels are tabulated in a 2×2 **Confusion Matrix**:

Actual / Predicted	Predicted Positive ($\hat{Y} = 1$)	Predicted Negative ($\hat{Y} = 0$)
Actual Positive ($Y = 1$)	True Positive (TP) Correctly identified positive	False Negative (FN) Type II Error (Miss)
Actual Negative ($Y = 0$)	False Positive (FP) Type I Error (False Alarm)	True Negative (TN) Correctly identified negative

1. **Classification Accuracy:** Proportion of total correct predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- **Accuracy Paradox:** On imbalanced datasets (e.g., 99% negative, 1% positive), a trivial model predicting negative always achieves 99% accuracy while failing entirely on positive instances.

2. **Precision (Positive Predictive Value):** Exactness of positive predictions:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- High precision is critical when False Positives are catastrophic (e.g., Spam filtering, YouTube video demonetization).

3. **Recall (Sensitivity / True Positive Rate - TPR):** Completeness of positive identification:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- High recall is critical when False Negatives are catastrophic (e.g., Malignant tumor detection, Earthquake warning).

4. **Specificity (True Negative Rate - TNR):**

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

5. **F1-Score:** Harmonic mean of precision and recall, balancing both metrics on imbalanced data:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}$$

6. **Receiver Operating Characteristic (ROC) & AUC:**

- **ROC Curve:** Plots True Positive Rate (TPR = Recall) on Y-axis against False Positive Rate (FPR = $1 - \text{Specificity} = \frac{\text{FP}}{\text{TN} + \text{FP}}$) on X-axis across all discrimination thresholds.
- **Area Under Curve (AUC):** Aggregate ranking measure between 0.5 (random guess) and 1.0 (perfect discrimination).

Regression Evaluation Metrics

1. **Mean Absolute Error (MAE):** Average magnitude of absolute errors:

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|$$

- Linear penalty; robust to extreme outliers.

2. **Mean Squared Error (MSE):** Average squared error penalty:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

- Quadratically penalizes large errors; sensitive to outliers.

3. **Root Mean Squared Error (RMSE):** Square root of MSE; expressed in the identical units of target variable y :

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2}$$

4. **Coefficient of Determination (R^2 Score):** Proportion of target variance explained by inputs:

$$R^2 = 1 - \frac{\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^m (y^{(i)} - \bar{y})^2} = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}$$

- $R^2 = 1.0$: Perfect fit. $R^2 = 0.0$: Equivalent to baseline mean predictor. $R^2 < 0$: Model performs worse than the sample mean.

Bias-Variance Decomposition & Overfitting/Underfitting

The generalization error of any supervised learning model decomposes into three distinct mathematical components:

$$\text{Expected Generalization Error} = \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2$$

Underfitting (High Bias)

- High Training Error
- High Validation Error
- Model too simplistic
- Remedy: Add features, increase model capacity

Optimal Balance

- Low Training Error
- Low Validation Error
- Minimal Generalization Gap
- Goal of Model Selection

Overfitting (High Variance)

- Very Low Training Error
- High Validation Error
- Model memorizes noise
- Remedy: Regularization, more data, pruning

Figure 3.5: Bias-Variance Tradeoff vs Model Complexity and Diagnostic Signatures

1. **Bias Error (Bias^2):** Error resulting from erroneous or oversimplified assumptions in the learning algorithm. High bias causes the model to miss underlying trends (**Underfitting**).
2. **Variance Error (Variance):** Error resulting from extreme sensitivity to small fluctuations in the training dataset. High variance causes the model to fit random noise (**Overfitting**).
3. **Irreducible Error (σ^2):** Inherent noise and unobserved variables in the data-generating distribution that no model can eliminate.

Cross-Validation & Resampling Strategies



Figure 3.6: K-Fold Cross-Validation Scheme (5-Fold Iterative Partitioning)

1. Holdout Validation Method:

- Split dataset randomly into **Training Set** (e.g., 70%), **Validation Set** (e.g., 15% for hyperparameter tuning), and **Test Set** (e.g., 15% for final evaluation).
- High variance in evaluation if the dataset is small or if unrepresentative samples fall into the test split.

2. K-Fold Cross-Validation:

- Split training dataset randomly into K equal-sized disjoint partitions (folds).
- Iterate K times: train on $K - 1$ folds and validate on the remaining single fold.
- Aggregate overall performance estimate: $CV_{(K)} = \frac{1}{K} \sum_{k=1}^K \text{Score}_k$. Typical values: $K = 5$ or $K = 10$.

3. Stratified K-Fold Cross-Validation:

- Guarantees that each fold preserves the identical percentage ratio of target class labels as the original complete dataset. Essential for imbalanced classification tasks.

4. Leave-One-Out Cross-Validation (LOOCV):

- Extreme case of K-Fold where $K = m$ (number of total instances).
- In each of m iterations, train on $m - 1$ points and test on exactly 1 point.
- Deterministic (no random split variance) and nearly unbiased, but computationally expensive for large datasets ($\mathcal{O}(m)$ model fits).

Source: Alpaydin, Ch 19, p. 486-495 | Source: Marsland, Ch 2, p. 35-42 | Source: Müller & Guido, Ch 5, p. 252-305